

NIYATI BHANANI

EPIC RPG PDF-V3

 MCA 4 Projects - 2025-26

Document Details

Submission ID

trn:oid:::3618:128144082

Submission Date

Feb 13, 2026, 7:56 AM GMT+5:30

Download Date

Feb 13, 2026, 7:59 AM GMT+5:30

File Name

EPIC RPG PDF-V3.pdf

File Size

4.1 MB

20 Pages

2,215 Words

12,087 Characters

52% detected as AI

The percentage indicates the combined amount of likely AI-generated text as well as likely AI-generated text that was also likely AI-paraphrased.

Caution: Review required.

It is essential to understand the limitations of AI detection before making decisions about a student's work. We encourage you to learn more about Turnitin's AI detection capabilities before using the tool.

Detection Groups

-  **23 AI-generated only 52%**
Likely AI-generated text from a large-language model.
-  **0 AI-generated text that was AI-paraphrased 0%**
Likely AI-generated text that was likely revised using an AI-paraphrase tool or word spinner.

Disclaimer

Our AI writing assessment is designed to help educators identify text that might be prepared by a generative AI tool. Our AI writing assessment may not always be accurate (i.e., our AI models may produce either false positive results or false negative results), so it should not be used as the sole basis for adverse actions against a student. It takes further scrutiny and human judgment in conjunction with an organization's application of its specific academic policies to determine whether any academic misconduct has occurred.

Frequently Asked Questions

How should I interpret Turnitin's AI writing percentage and false positives?

The percentage shown in the AI writing report is the amount of qualifying text within the submission that Turnitin's AI writing detection model determines was either likely AI-generated text from a large-language model or likely AI-generated text that was likely revised using an AI paraphrase tool or word spinner.

False positives (incorrectly flagging human-written text as AI-generated) are a possibility in AI models.

AI detection scores under 20%, which we do not surface in new reports, have a higher likelihood of false positives. To reduce the likelihood of misinterpretation, no score or highlights are attributed and are indicated with an asterisk in the report (*%).

The AI writing percentage should not be the sole basis to determine whether misconduct has occurred. The reviewer/instructor should use the percentage as a means to start a formative conversation with their student and/or use it to examine the submitted assignment in accordance with their school's policies.

What does 'qualifying text' mean?

Our model only processes qualifying text in the form of long-form writing. Long-form writing means individual sentences contained in paragraphs that make up a longer piece of written work, such as an essay, a dissertation, or an article, etc. Qualifying text that has been determined to be likely AI-generated will be highlighted in cyan in the submission, and likely AI-generated and then likely AI-paraphrased will be highlighted purple.

Non-qualifying text, such as bullet points, annotated bibliographies, etc., will not be processed and can create disparity between the submission highlights and the percentage shown.



1. SYNOPSIS:

- The Text-Based Python RPG Game is a command-line game developed using Python, where players can register, log in, create their character, explore different areas, fight enemies, collect items, and gain experience as the game progresses.
- The gameplay is intentionally kept simple so that both playing the game and understanding how it is built feels easy and enjoyable.
- The project is structured in a modular way, with different files handling specific parts of the game such as user management, game logic, inventory system, store operations, and utility functions. This approach keeps the code clean and well-organized.
- Player data like username, password, character stats, experience points, coins, and inventory is stored using JSON files, allowing the game to save progress and continue smoothly.
- The main motive behind this project was to make gaming easy and to show that game development does not always require complex graphics or advanced tools. Through this project, it is demonstrated that a complete and engaging game can be developed using simple logic and basic Python concepts.
- The focus was not only on the final game but also on making the process of game development understandable and enjoyable.
- Being a text-based game, it can run on any system with Python installed and does not require special hardware. This project clearly shows how fundamental programming concepts such as functions, loops, conditional statements, file handling, and modular coding can be used to build a real-world application.
- Overall, the project helped strengthen logical thinking, creativity, and confidence in developing games and applications in a simple yet effective way.

2. PREAMBLE

2.1 General Introduction:

- Text-based RPGs (Role-Playing Games) are one of the earliest forms of computer games, built entirely through text prompts and user choices. These games are simple yet highly interactive, relying on decision-making rather than graphics.

2.2 Statement of Problem:

- Many beginner programmers find it difficult to understand how real games manage player progress, combat, and rewards. Most examples available are either too simple to be engaging or too complex to understand. This project aims to solve that problem by creating a simple, text-based RPG using basic Python, where core game mechanics like combat, crafting, inventory, and progression are easy to follow, modify, and learn from

2.3 Objective and Scope of the study:

- Implement user registration and login functionality.
- Created a system for character level progression and attribute management (health, XP, coins).
- Design and developed various game modules, including hunting, adventure, inventory management, a store, and character profiles.
- Incorporate file-based storage for user data persistence.
- Provide a user-friendly text-based interface with clear instructions and feedback.
- **Scope:**
 - The project will include core RPG elements such as character development, combat, and inventory.
 - The game will be text-based, focusing on gameplay mechanics rather than graphical presentation.
 - User data will be stored in a JSON file.
 - The game will be developed using Python 3.x.

2.4 Module Description with functionality:

2.4.1 User Management Module:

- Description: This module handles user registration, login, and deletion.
- Functions:
 - register(): Creates a new user account, storing the username and password.
 - login(): Authenticates existing users and retrieves their data.
 - delete_user(): Removes a user account from the system.
 - load_user_data(): Loads user data from the JSON file.
 - save_user_data(): Saves user data to the JSON file.

2.4.2 Game Mechanics Module:

- Description: This module contains the core gameplay functions, including hunting, adventure, chopping, and reward systems.
- Functions:
 - rpg_hunt(): Simulates a hunting expedition where the player encounters creatures, gains rewards, and loses health.
 - rpg_adventure(): Presents a more challenging adventure scenario with a single, powerful creature.
 - rpg_chop(): Allows the player to chop wood and gather resources.
 - reward(): Provides random rewards to the player.
 - get_level(): Calculates the player's level based on their XP.
 - check_level_up(): Checks if the player has leveled up and updates their stats

2.4.3 Inventory Module:

- Description: Manages the player's items.
- Function:
 - view_inventory(): Displays the items currently in the player's inventory.

2.4.4 Profile Module:

- Description: Displays the player's character information.
- Function:
 - view_profile(): Shows the player's username, level, stats (attack, defense, health), coins, and XP.

2.4.5 Store Module:

- Description: Allows the player to buy items from a store.
- Function:
 - store(): Displays the store's inventory and handles purchases.

2.4.6 Main Game Module:

- Description: The main entry point of the game, orchestrating the game loop and user interaction.
- Function:
 - start_game(): Manages the main game loop, user input, and calls other modules.

2.4.7 Utils Module:

- Description: Provides utility functions and data structures used across other modules.
- Functions/Data:
 - get_level(): Calculates the player's level.
 - item_emojis: A dictionary mapping item names to emojis for display.

3. Review of Literature

3.1 Review of Literature :- Basic Theory of the Game

- Role-playing games work on the idea that a player starts with limited abilities and improves over time by playing the game. As the player performs actions like fighting enemies or collecting items, they gain experience, level up, and become stronger.
- Most RPGs use concepts such as health, experience points, inventory, and equipment to show player progress. This project follows the same basic idea but keeps everything text-based so the logic behind the game is easy to understand and learn

3.2 Study of Existing Similar Systems :- Zork

- One of the earliest examples of a similar game is Zork, a popular text-based adventure game released in the late 1970s. In Zork, players explored a fantasy world by typing commands, solving puzzles, and collecting items.
- It did not use graphics, but still managed to be engaging and immersive. This project takes inspiration from games like Zork by using text interaction, while also adding modern RPG features such as combat, levelling, crafting, and inventory management. The focus is on simplicity and learning rather than complex graphics.

4. Technical Description

4.1 Hardware Requirements

- Processor: Intel / AMD processor or equivalent
- RAM: Minimum 2 GB (4 GB or more recommended)
- Storage: 200–300 MB free disk space to support future game data, save files, logs, and feature expansions
- Display: Terminal or console window
- Input Device: Keyboard

4.2 Software Requirement

- Windows: Windows 10 or later
- Linux: Modern Linux distributions
- macOS: macOS 11 or later

5. System Design and Development

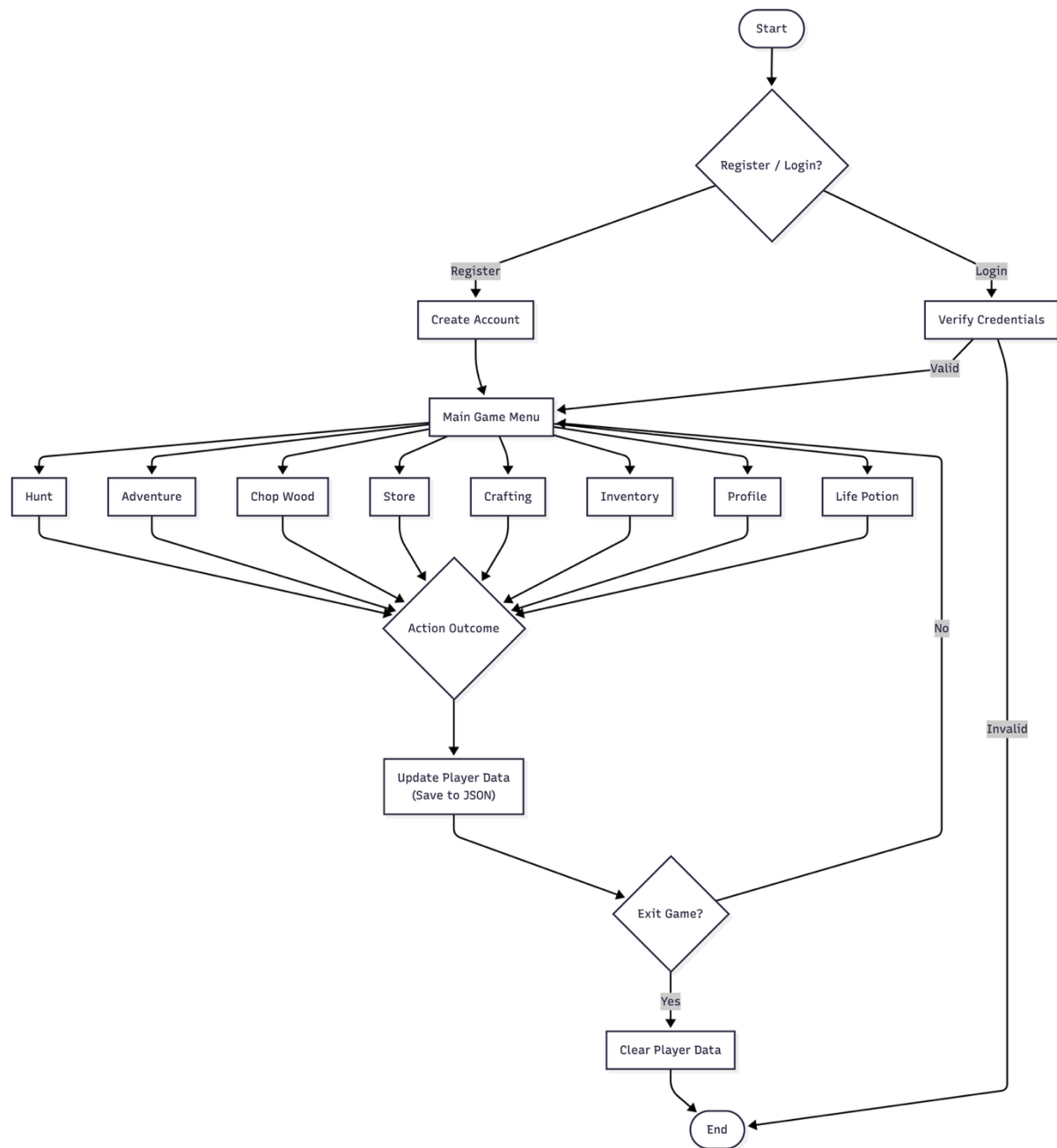


Figure 5.1 Flowchart:

Epic RPG

Data flow diagram:

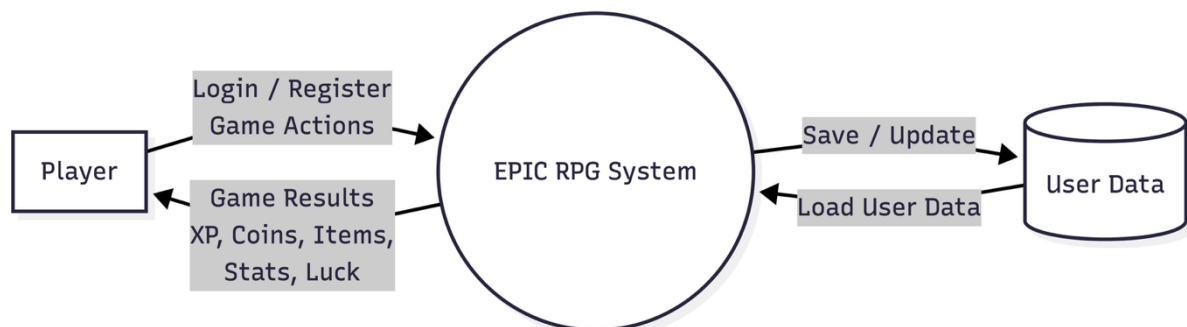


Figure 5.2 DFD-0

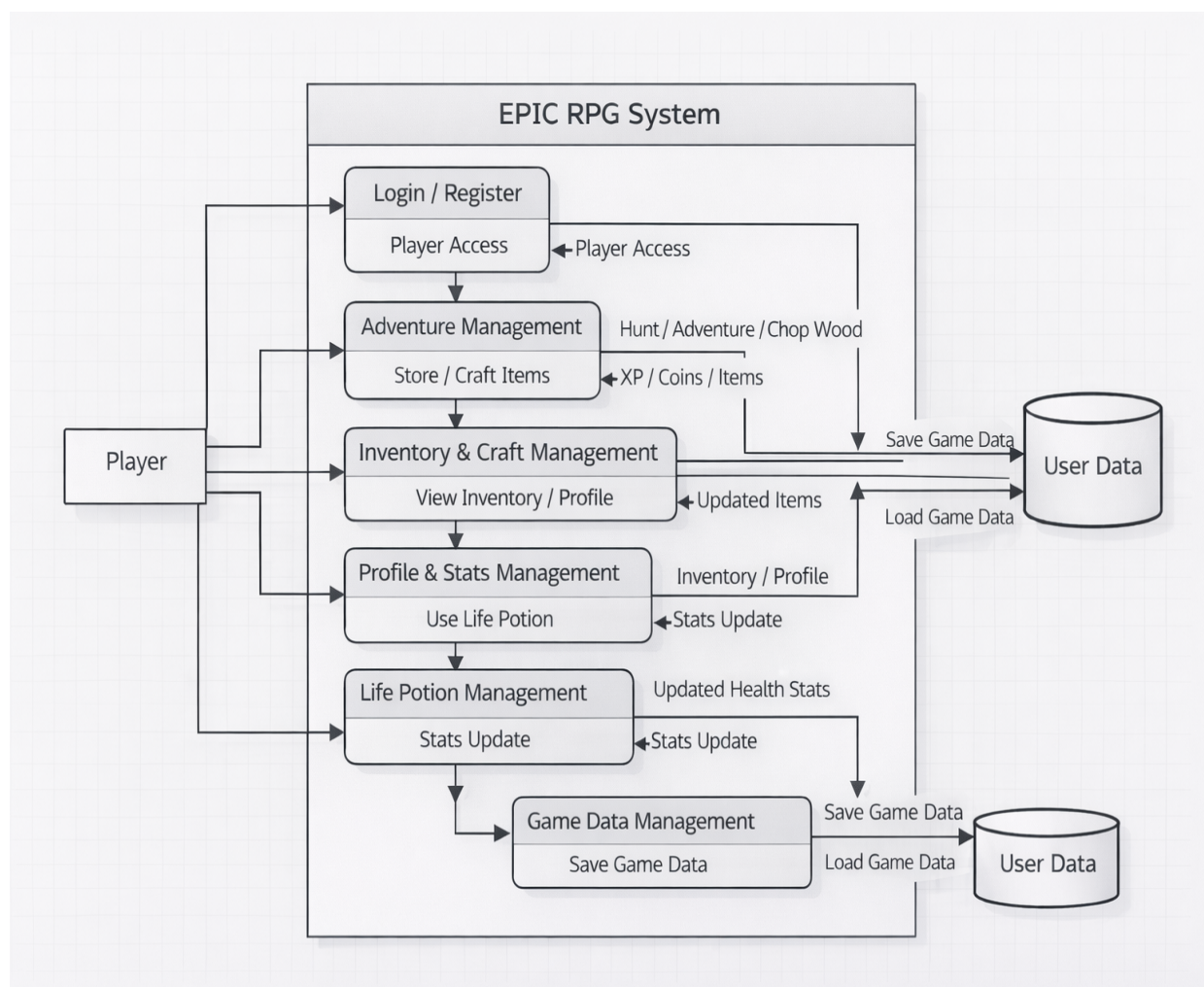


Figure 5.3 DFD Level-1

Epic RPG

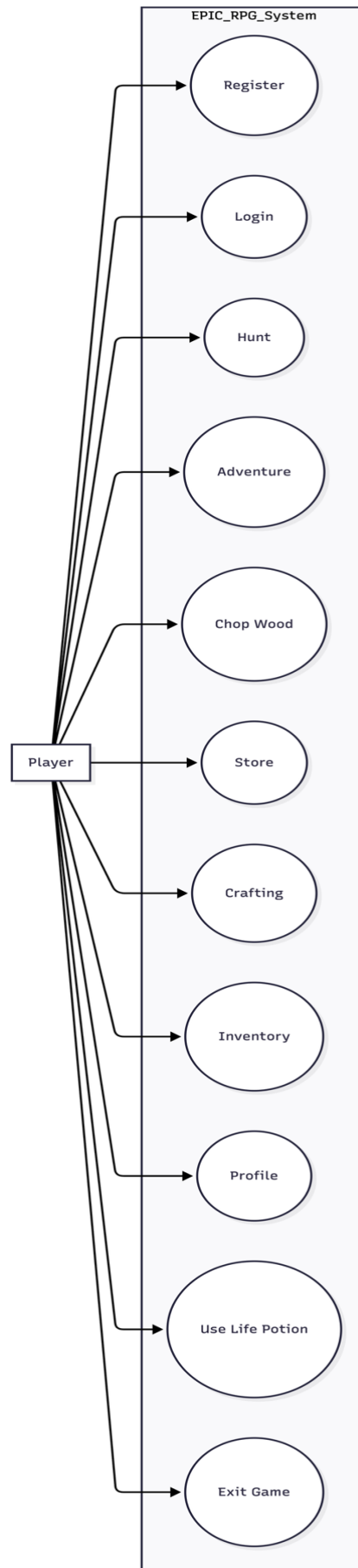


Figure 5.4 Use case diagram

Epic RPG

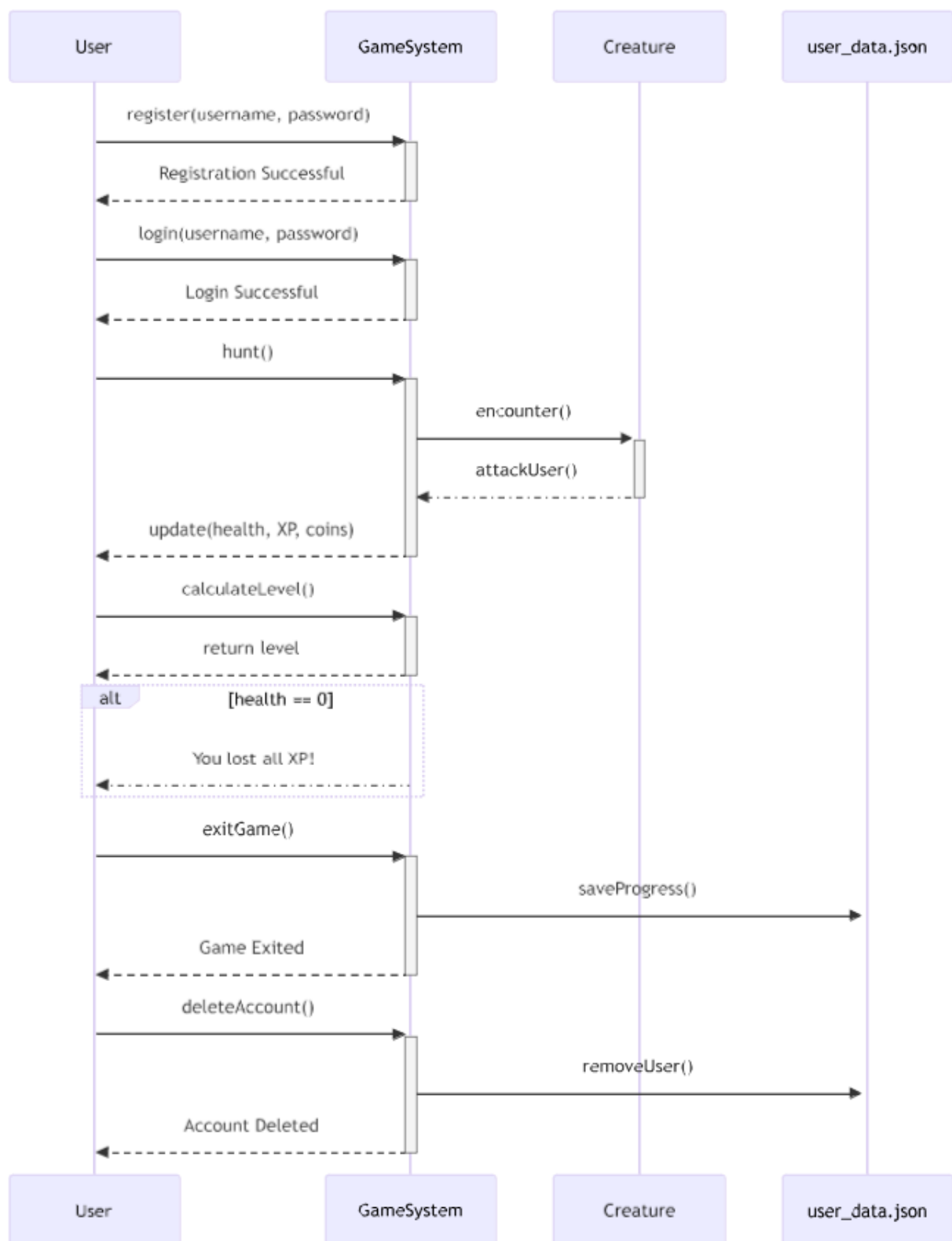


Figure 5.5 Sequential diagram

Epic RPG

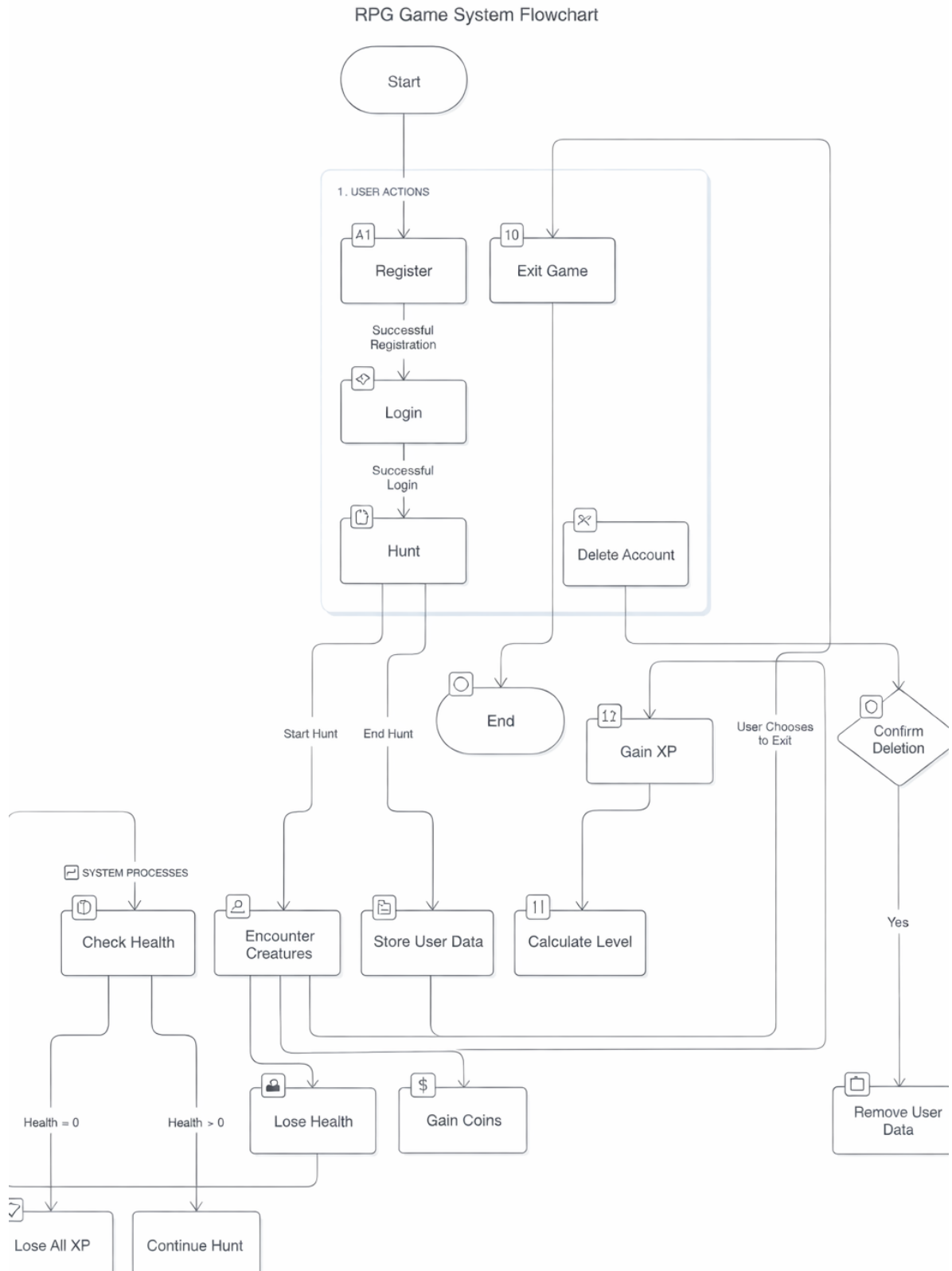


Figure 5.6 Activity diagram:

Epic RPG

5.1 Menu Design

- The menu is the main place where the player controls the game. It shows all the available actions in a simple numbered list so the player can easily decide what to do next.
- The player just enters the number of the option they want, and the game performs that action. After the action is completed, the menu is shown again so the player can continue playing without any confusion.

MENU

1. 🎯 Hunt
2. 🗺️ Adventure
3. 🍷 Heal using Life Potion
4. 📁 View Inventory
5. 👤 View Profile
6. 🏪 Store
7. 🪓 Chop Wood
8. 🎁 Reward
9. 🛠️ Craft
10. 🗑️ Delete User
11. 🚪 Exit

Epic RPG

5.2 Screen Design

```
adil@192 RPG % python3 main.py

Menu:
1. Register
2. Login
Enter your choice: 2
Enter your username: a
Enter your password: a
Login successful! Welcome, a!

Menu:
1. 🎯 Hunt
2. 🗺️ Adventure
3. 🥬 Heal using Life Potion
4. 🎒 View Inventory
5. 👤 View Profile
6. 🏪 Store
7. 🪵 Chop Wood
8. 📺 Reward
9. 🛠️ Craft
10. 🗑️ Delete User
11. 🚪 Exit
Enter your choice: █
```

Figure 5.7 Login/Register/Main Menu

Epic RPG

Hunt:

- During gameplay, the player hunts a deer enemy. The system updates health, XP, coins, and inventory in real time. Successful encounters contribute to player progression.

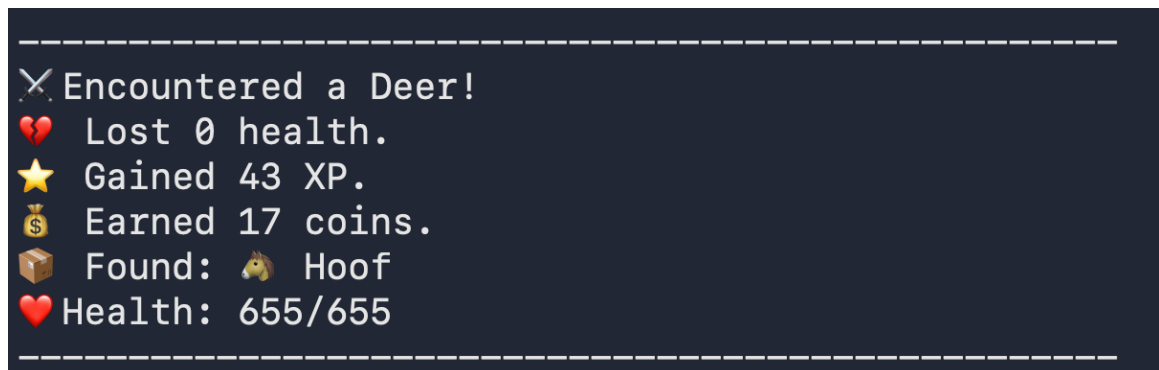


Figure 5.8 Hunt

Adventure:

- The player fights a Demonic Hydra, takes damage, and defeats the enemy. Rewards include XP, coins, and a level increase.

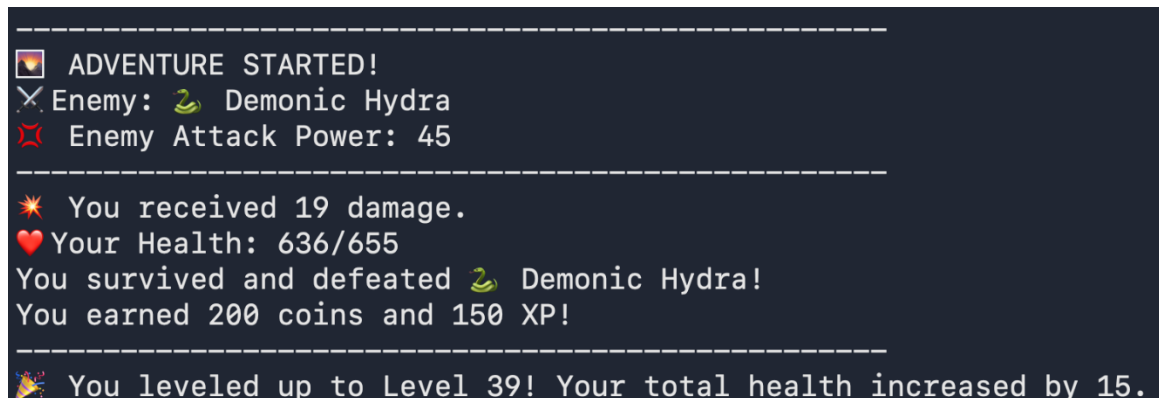


Figure 5.9 Adventure

Epic RPG

Inventory:

- The inventory module maintains a list of all items obtained from battles and exploration. Each item is stored with quantity values and used for healing, crafting, or combat enhancement.

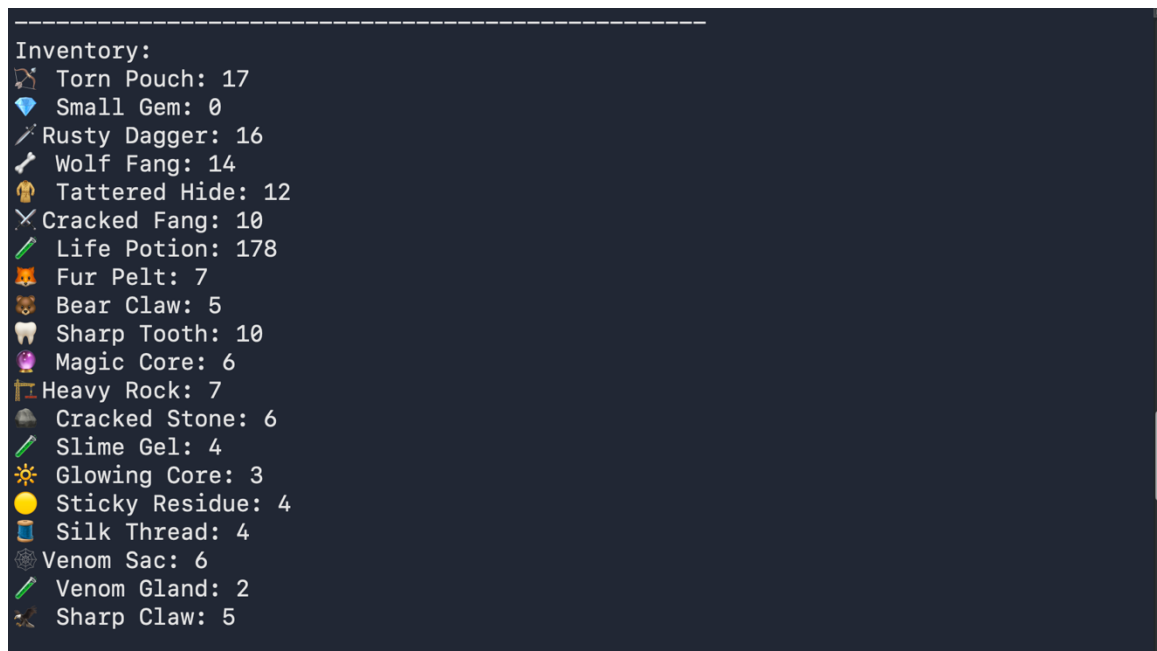


Figure 5.10 Inventory

Profile:

- The player profile module maintains real-time character attributes such as combat stats, equipment, and resources. All values are dynamically updated based on gameplay actions.

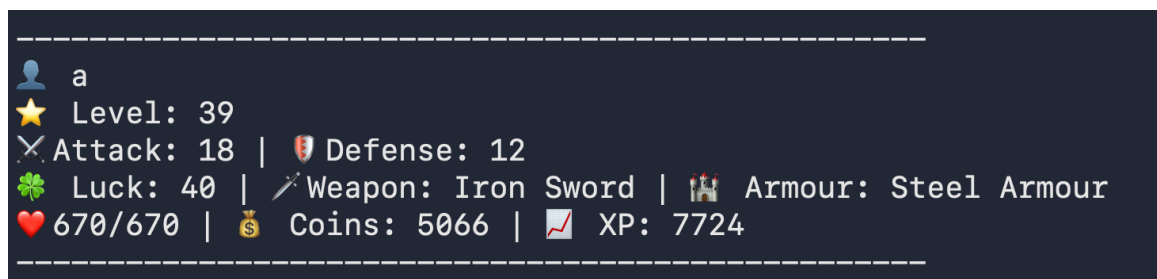


Figure 5.11 Profile

Epic RPG

Store:

- The player can buy weapons, armour, and life potions from the store. Coins are deducted based on the selected item.

```
Store:
💰 Coins: 5066
1. 🪓🗡️ Wooden Sword: 100 coins
2. 🗡️🔗 Iron Sword: 300 coins
3. 🛡️ Basic Armour: 200 coins
4. 🛡️🔗 Steel Armour: 400 coins
5. 🧴 Life Potion: 20 coins
```

Figure 5.12 Store

Chop:

- The player gathers resources by interacting with the environment. Obtained items are added to the inventory.

```
🪓 You chopped down a tree and obtained: 🌲 Pine Log
```

Figure 5.13 Chop

Reward:

- Reward system showing loot distribution after successful completion of an action.

```
🎁 You received a reward!
💰 Coins: 396
🧴 Life Potions: 6
```

Figure 5.14 Reward

Craft:

- The player crafts new items using collected resources. Each item requires specific materials.

```
Crafting:
1. Wooden Sword (needs 5 🪓 Oak Logs)
2. Lucky Charm (needs 3 💎 Small Gems)
3. Exit
```

Figure 5.15 Craft

6. System Testing

Epic RPG

6.1 Testing and Implementation

- Testing was done throughout the development of the project by running the game in the terminal after adding new features. Each time a feature was implemented, the game was played to check if it was working properly.
- When errors occurred, such as crashes, wrong outputs, or logical mistakes, the code was reviewed and fixed immediately. This ongoing testing helped keep the game stable while new features were being added.

6.2 Testing Methodology

- Manual testing was followed for this project. Since the game runs entirely in the command line, testing was done by selecting different menu options and observing how the system responds.
- Various situations were tested, including normal gameplay, invalid inputs, missing inventory items, cooldown limits, and login failures. This helped ensure that the game handles user actions correctly and does not break during gameplay.

6.3 Test Case Design

- Test cases were created based on the options available in the game menu. Each test case focused on checking one feature at a time and observing how player data such as health, experience points, inventory items, and overall statistics were updated during gameplay.

Test Case ID	Menu Option	Description	Expected Result
TC-01	Register	Create a new user account	User account created successfully
TC-02	Login	Login with valid credentials	User logged in
TC-03	Login	Login with invalid credentials	Access denied
TC-04	Hunt	Perform hunt action	XP, coins, and items updated
TC-05	Adventure	Perform adventure	Health reduced or rewards gained
TC-06	Chop Wood	Chop wood	Wood logs added to inventory
TC-07	Store	Buy weapon or item	Inventory updated, stats affected
TC-08	Craft	Craft Wooden Sword	Weapon crafted, attack increased
TC-09	Craft	Craft Lucky Charm	Luck increased, cooldown applied
TC-10	Heal	Use Life Potion	Health restored
TC-11	Inventory	View inventory	Items displayed correctly
TC-12	Profile	View profile	Player stats displayed
TC-13	Reward	Claim reward	XP or coins added
TC-14	Delete User	Delete account	User data removed
TC-15	Exit	Exit game	Program terminates safely

Table No. 6.1

6.4 System Implementation Strategy

- The system was implemented in a step-by-step manner to keep the development process simple and manageable. Basic features such as user registration, login, and menu navigation were implemented first.
- Once these core features were stable, gameplay functions like hunting, adventure, inventory handling, and profile viewing were added.
- Advanced features such as the store, crafting system, cooldown mechanisms, and luck attribute were implemented later. After adding each feature, the system was tested through the terminal to ensure correct behaviour.
- This gradual implementation approach made it easier to identify errors, apply fixes, and maintain overall stability of the game.

7. Conclusion

- This project helped in understanding how a role-playing game works at a basic level using Python. By building a text-based RPG, important programming concepts such as functions, data handling, file storage, and logic flow were learned and applied in a practical way.
- Features like combat, inventory management, crafting, and player progression were implemented step by step, keeping the system simple and easy to understand.
- Working on this project also improved problem-solving skills, as many errors were encountered during development and fixed through testing and debugging. Overall, the project successfully meets its objective of creating a simple yet functional RPG system while providing a good learning experience in Python programming.

8. Learning During Project Work

- Working on this project was a valuable learning experience. It helped in understanding how to break a large problem into smaller parts and implement them step by step.
- Through this project, practical knowledge of Python programming was gained, especially in areas like file handling, functions, dictionaries, and program flow control.
- The project also improved problem-solving skills, as many errors and logical issues were faced during development and resolved through testing and debugging. Additionally, it provided experience in designing a system, planning features, and improving code gradually.
- Overall, this project helped build confidence in programming and gave a better understanding of how real applications are developed.

9. Online References

- [1] W3Schools, "Python Tutorial," W3Schools. [Online]. Available: <https://www.w3schools.com/python/>. Accessed throughout the project.
- [2] EPIC RPG Wiki, "EPIC RPG Game Guide," Fandom. [Online]. Available: https://epic-rpg.fandom.com/wiki/EPIC_RPG_Wiki. Accessed throughout the project.

This project marks the end of an important phase of my academic journey. Building this game was not just about writing code, but about learning, experimenting, failing, fixing mistakes, and improving step by step. It reflects the effort, patience, and growth I experienced throughout my final year, making this project a memorable and meaningful conclusion to my studies.